Министерство науки и высшего образования Российской Федерации
ФГАОУ ВО «Уральский федеральный университет
имени первого Президента России Б.Н. Ельцина»
Институт новых материалов и технологий
Кафедра «Теплофизика и информатика в металлургии»

Использование конвейера данных реального времени с Kafka

ОТЧЕТ
по практической работе № 3
по дисциплине «Основы методологии Development Operation»

Направление 09.03.02 «Информационные системы и технологии» (уровень бакалавриата)
Образовательная программа
09.03.02/33.02 «Информационные системы и технологии» (СУОС)

Студент
группы НМТ-433901 О.М. Малышева

Преподаватель:
профессор, д.т.н. В.В. Лавров

Екатеринбург
2026

СОДЕРЖАНИЕ

Практическая работа «Использование конвейера данных реального времени с Kafka» 3
1.1 Цель работы 3
1.2 Ход проведения работы 3
1.2.1 Разработка тестового приложения Visual Studio .NET Core 3
1.2.2 Размещение приложения в системе контроля версий GitHub 3
1.2.3 Сборка образов 3
1.2.4 Установка программы HeidiSQL для сопровождения базы данных в СУБД MariaDB ... 4
1.2.5 Настройка миграции базы данных в MariaDB 5
1.2.6 Демонстрация работы приложения брокером сообщений Kafka 5
1.3 Выводы 6
Приложение А Листинг программного кода приложения «Калькулятор» (контроллер) 8
Приложение Б Листинг программного кода приложения «Калькулятор» (KafkaConsumerService.cs) 11
Приложение В Листинг программного кода приложения «Калькулятор» (appsettings.json) 13
Приложение Г Листинг программного кода приложения «Калькулятор» (appsettings.Development.json) 14
Приложение Д Листинг программного кода приложения «Калькулятор» (Program.cs) 15

Практическая работа
«Использование конвейера данных реального времени с Kafka»

1.1 Цель работы

Цель практической работы – освоение принципов построения конвейера обработки данных реального времени с использованием брокера сообщений Apache Kafka, а также получение практических навыков интеграции приложения ASP.NET Core с Kafka и организации передачи и получения сообщений.

1.2 Ход проведения работы

1.2.1 Разработка тестового приложения Visual Studio .NET Core

В качестве тестового приложения использован веб-калькулятор, разработанный на платформе ASP.NET Core. Приложение выполняет четыре арифметические операции: сложение, вычитание, умножение и деление. Перед выполнением деления предусмотрена проверка делителя на равенство нулю.

После вычисления результат сохраняется в базе данных MariaDB. Далее формируется сообщение, содержащее два операнда, тип операции, результат и временную метку UTC. Данные сериализуются в формат JSON и передаются в Kafka.

Для взаимодействия с брокером в контроллере используется библиотека Confluent.Kafka. Отправитель создаётся с параметром BootstrapServers, получает имя топика из конфигурации и отправляет сообщение с уникальным ключом.

1.2.2 Размещение приложения в системе контроля версий GitHub

Исходный код приложения и файлы конфигурации размещены в репозитории GitHub. Это позволяет хранить версии проекта и переносить исходные файлы на сервер для сборки и запуска контейнеризированного приложения.

1.2.3 Сборка образов

Для запуска приложения используется Docker Compose. В конфигурации определён сервис web с именем контейнера calculator_olesya_5007. Образ собирается из каталога ./Calculator, а порт 5007 контейнера публикуется на порту 5007 сервера.

Контейнер приложения подключён к двум внешним Docker-сетям: mariadbnet для взаимодействия с MariaDB и kafkanet для подключения к Kafka. Адрес брокера передаётся приложению через переменную окружения Kafka__BootstrapServers со значением kafka-kafka-1:9092. Для запуска приложения используется Docker Compose. В конфигурации определён сервис web с именем контейнера calculator_olesya_5007. Образ собирается из каталога ./Calculator, а порт 5007 контейнера публикуется на порту 5007 сервера.

Контейнер приложения подключён к двум внешним Docker-сетям: mariadbnet для взаимодействия с MariaDB и kafkanet для подключения к Kafka. Адрес брокера передаётся приложению через переменную окружения Kafka__BootstrapServers со значением kafka-kafka-1:9092.

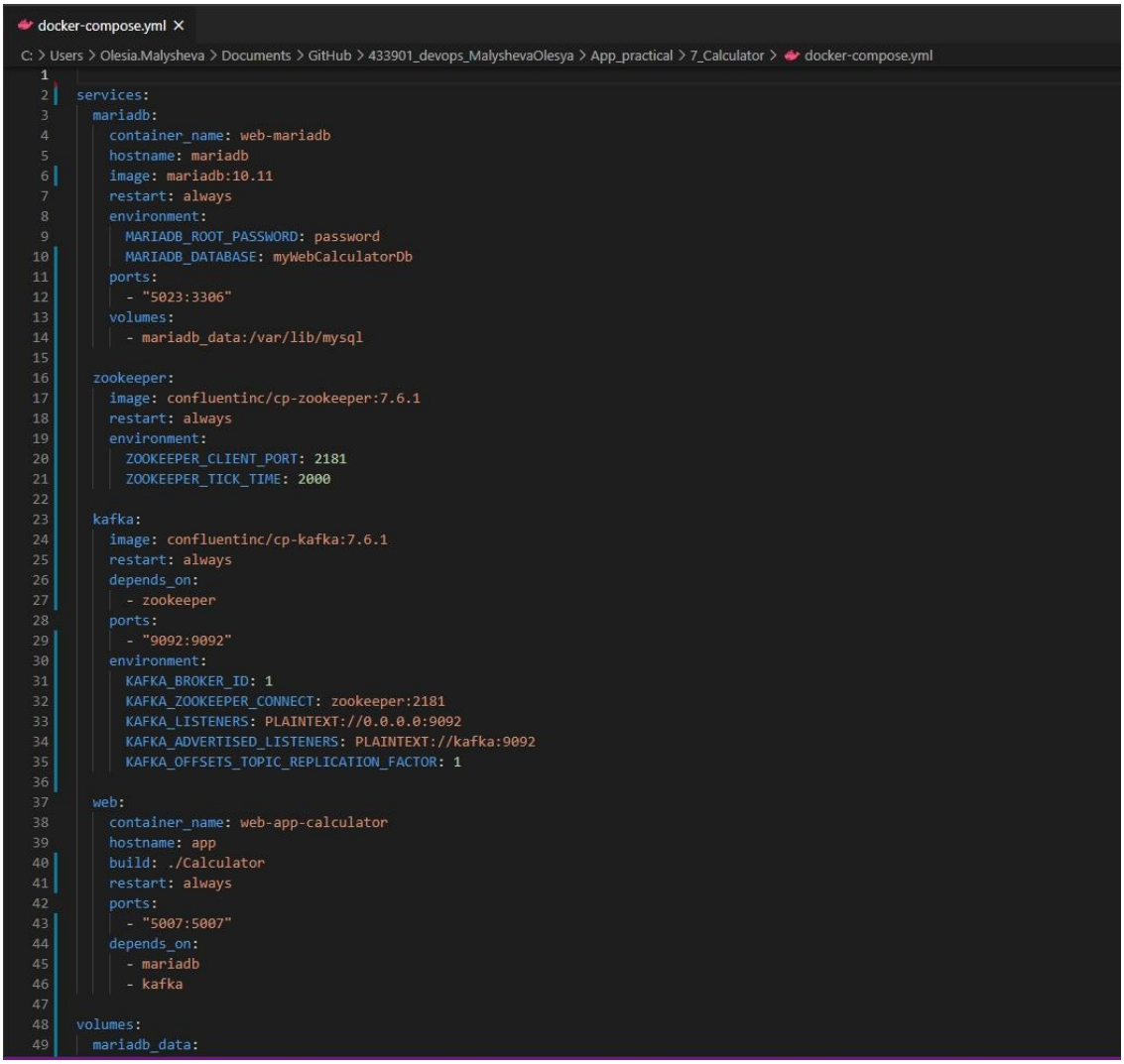


Рисунок 1 – Конфигурация Docker Compose для приложения-калькулятора

1.2.4 Установка программы HeidiSQL для сопровождения базы данных в СУБД MariaDB

Для просмотра содержимого базы данных использована программа HeidiSQL. После подключения к серверу MariaDB была открыта база calculator_db. В ней отображаются таблицы __EFMigrationsHistory и Calculations, что подтверждает наличие структуры базы данных и таблицы, предназначенной для хранения истории вычислений.

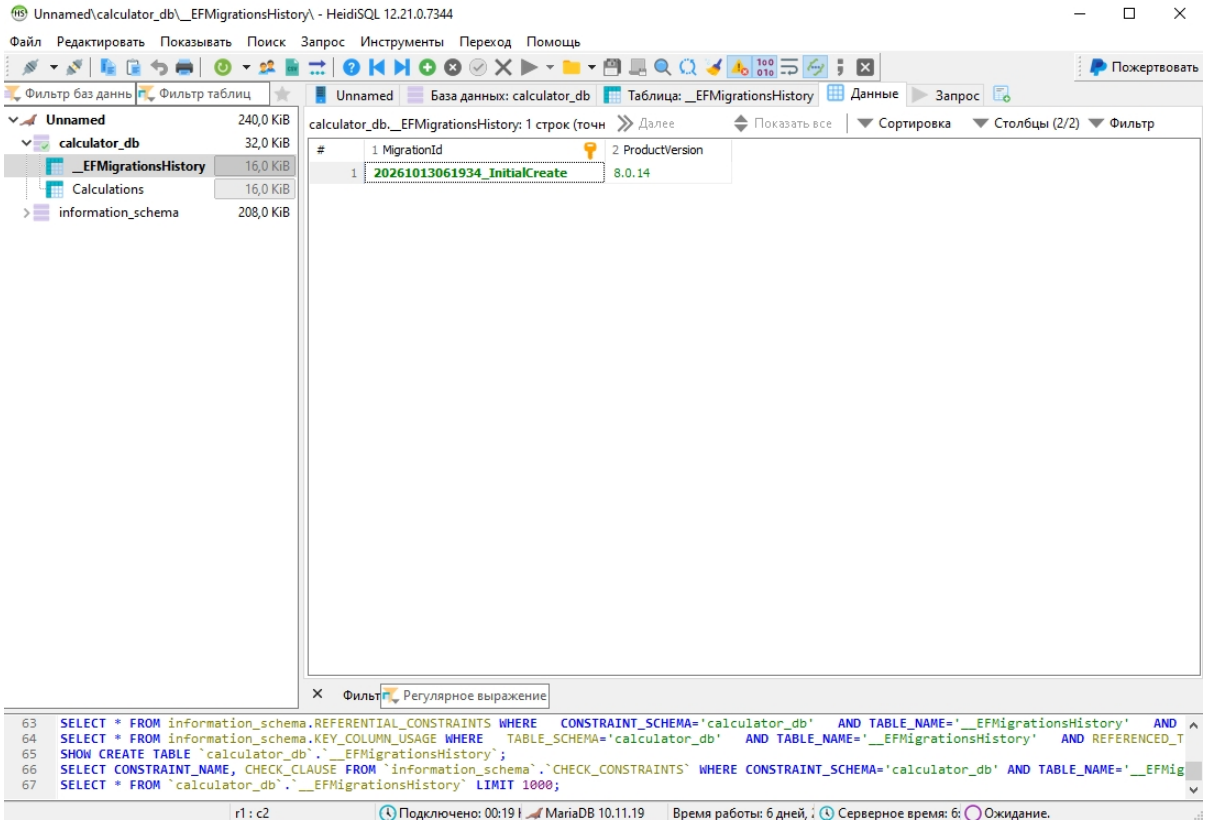


Рисунок 2 – База calculator_db и таблица __EFMigrationsHistory в HeidiSQL

1.2.5 Настройка миграции базы данных в MariaDB

Для создания структуры базы данных используется механизм миграций Entity Framework Core. В таблице __EFMigrationsHistory отображается запись InitialCreate и версия продукта 8.0.14. Это свидетельствует о том, что миграция была зарегистрирована в базе данных.

Проверка таблицы миграций и таблицы Calculations в HeidiSQL позволяет убедиться, что структура базы данных создана и доступна для приложения.

1.2.6 Демонстрация работы приложения брокером сообщений Kafka

В приложении реализована отправка вычислений в топик malysheva. После выполнения операции контроллер сериализует сформированный объект в JSON и вызывает ProduceAsync(). При успешной доставке в журнал приложения записываются имя топика, номер раздела и смещение сообщения.

Для приёма сообщений в приложении зарегистрирован фоновый сервис KafkaConsumerService. Он создаёт Kafka Consumer с группой calculator-consumer-group, подписывается на топик malysheva и непрерывно получает сообщения. Полученное содержимое выводится в журнал приложения.

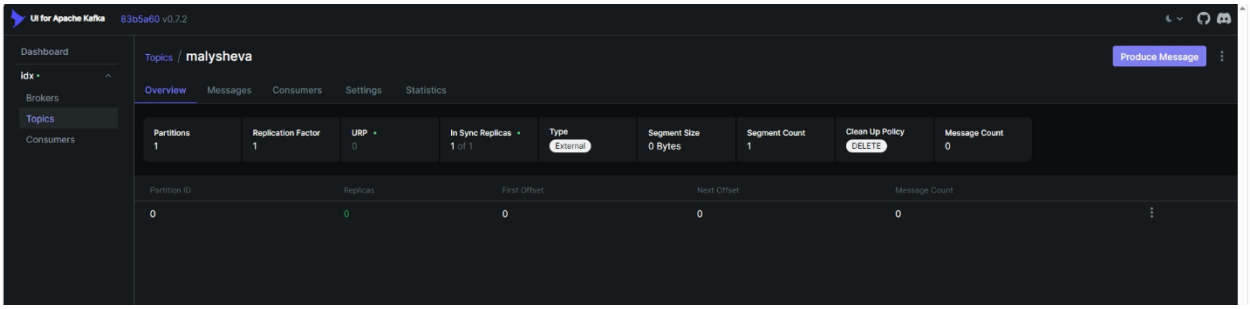


Рисунок 3 – Топик malysheva до отправки сообщений

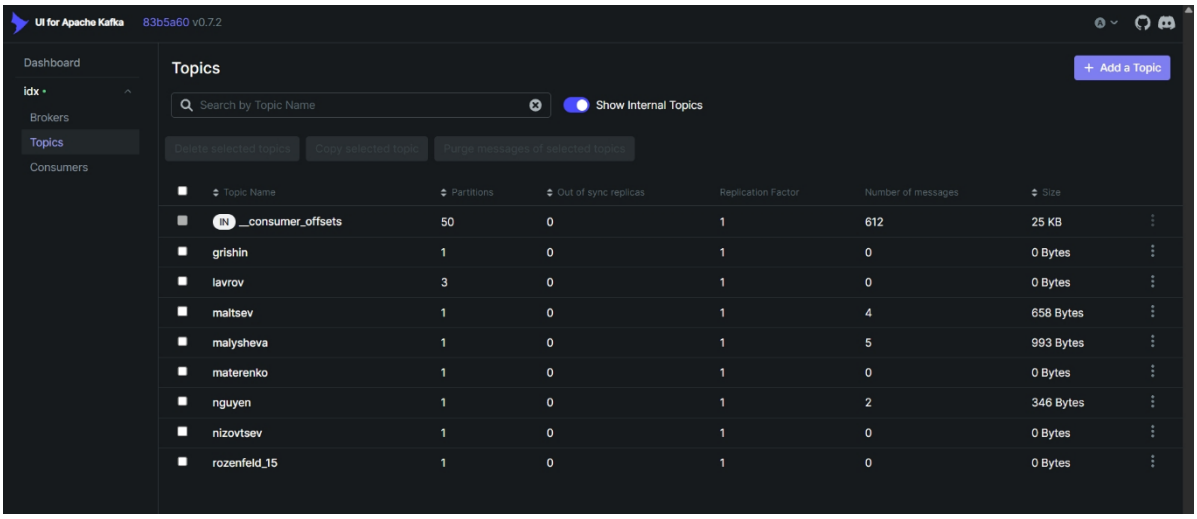


Рисунок 4 – Список топиков Kafka, включая malysheva

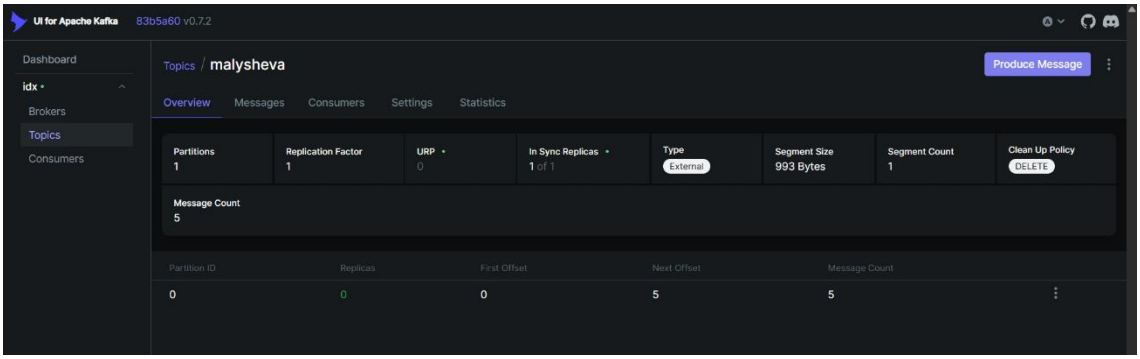


Рисунок 5 – Топик malysheva после отправки пяти сообщений

На странице управления Kafka отображается топик malysheva с одним разделом и пятью сообщениями. До выполнения проверочных операций количество сообщений составляло 0. После отправки вычислений счётчик увеличился до 5, что подтверждает поступление сообщений в брокер.

1.3 Выводы

В ходе практической работы выполнена интеграция веб-приложения-калькулятора с брокером сообщений Apache Kafka. Реализована отправка результатов арифметических вычислений в топик malysheva в формате JSON, а также получение сообщений отдельным фоновым

сервисом. Приложение подключено к Kafka через Docker-сеть, а конфигурация обмена сообщениями задаётся параметрами приложения. В интерфейсе управления Kafka подтверждено поступление пяти сообщений в созданный топик. Поставленная цель работы достигнута.

Приложение А Листинг программного кода приложения «Калькулятор» (CalculatorController.cs)

```
using Calculator.Data;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using Confluent.Kafka;
using System.Globalization;
using System.Text.Json;

namespace Calculator.Controllers
{
    public enum Operation { Add, Subtract, Multiply, Divide }

    public class CalculatorController : Controller
    {
        private readonly CalculatorContext _context;
        private readonly IConfiguration _configuration;
        private readonly ILogger<CalculatorController> _logger;

        public CalculatorController(
            CalculatorContext context,
            IConfiguration configuration,
            ILogger<CalculatorController> logger)
        {
            _context = context;
            _configuration = configuration;
            _logger = logger;
        }

        private void LoadHistory()
        {
            ViewBag.History = _context.DataInputVariants
                .AsNoTracking()
                .OrderByDescending(x => x.ID_DataInputVariant)
                .Take(20)
                .ToList();
        }

        [HttpGet]
        public IActionResult Index()
        {
            LoadHistory();
            return View();
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> Calculate(
            double num1,
            double num2,
            Operation operation)
        {
            if (operation == Operation.Divide && num2 == 0)
            {
                ModelState.AddModelError("", "На ноль делить нельзя.");
                LoadHistory();
                return View("Index");
            }
```

```
            double result = operation switch
            {
                Operation.Add => num1 + num2,
                Operation.Subtract => num1 - num2,
                Operation.Multiply => num1 * num2,
                Operation.Divide => num1 / num2,
                _ => throw new ArgumentOutOfRangeException(nameof(operation))
            };

            var calculation = new DataInputVariant
            {
                Operand_1 = num1,
                Operand_2 = num2,
                Type_operation = (int)operation,
                Result = result.ToString(
        System.Globalization.CultureInfo.InvariantCulture)
            };

            _context.DataInputVariants.Add(calculation);
            await _context.SaveChangesAsync();

            var message = new
            {
                Num1 = num1,
                Num2 = num2,
                Operation = operation.ToString(),
                Result = result,
                Timestamp = DateTime.UtcNow
            };

            try
            {
                var config = new ProducerConfig
                {
                    BootstrapServers =
                        _configuration["Kafka:BootstrapServers"],
                    MessageTimeoutMs = 5000
                };

                using var producer =
                    new ProducerBuilder<string, string>(config).Build();

                var json = JsonSerializer.Serialize(message);
                var topic = _configuration["Kafka:Topic"] ?? "malysheva";

                var delivery = await producer.ProduceAsync(
                    topic,
                    new Message<string, string>
                    {
                        Key = Guid.NewGuid().ToString(),
                        Value = json
                    });

                _logger.LogInformation(
                    "Kafka: сообщение отправлено в {Topic}, partition {Partition}, offset
{Offset}",
                    delivery.Topic,
                    delivery.Partition.Value,
```

```
                    delivery.Offset.Value);
            }
            catch (Exception ex)
            {
                _logger.LogError(ex, "Не удалось отправить вычисление в Kafka");
                ViewBag.KafkaError = "Вычисление сохранено, но отправка в Kafka не уда-
лась.";
            }

            ViewBag.Result = result;
            LoadHistory();
            return View("Index");
        }
    }
}
```

Приложение Б Листинг программного кода приложения «Калькулятор» (KafkaConsumerService.cs)

```
using Confluent.Kafka;
using Microsoft.Extensions.Hosting;

namespace Calculator.Services
{
    public class KafkaConsumerService : BackgroundService
    {
        private readonly ILogger<KafkaConsumerService> _logger;
        private readonly IConfiguration _configuration;

        public KafkaConsumerService(
            ILogger<KafkaConsumerService> logger,
            IConfiguration configuration)
        {
            _logger = logger;
            _configuration = configuration;
        }

        protected override async Task ExecuteAsync(
            CancellationToken stoppingToken)
        {
            var config = new ConsumerConfig
            {
                BootstrapServers = _configuration["Kafka:BootstrapServers"],
                GroupId = "calculator-consumer-group",
                AutoOffsetReset = AutoOffsetReset.Earliest,
                EnableAutoCommit = true
            };

            using var consumer = new ConsumerBuilder<string, string>(config).Build();

            consumer.Subscribe("malysheva");

            _logger.LogInformation("Kafka consumer запущен");

            try
            {
                while (!stoppingToken.IsCancellationRequested)
                {
                    var result = consumer.Consume(stoppingToken);
                    _logger.LogInformation(
                        "Kafka: получено сообщение: {Message}",
                        result.Message.Value);

                    await Task.Yield();
                }
            }
            catch (OperationCanceledException)
            {
                // Сервис останавливается
            }
            finally
            {
                consumer.Close();
            }
        }
```

```
    }
}
```

Приложение В Листинг программного кода приложения «Калькулятор» (appsettings.json)

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=mariadb;Database=myWebCalculatorDb;Uid=root;Pwd=password;Character Set=utf8;ConvertZeroDatetime=True;"
  },
  "Kestrel": {
    "Endpoints": {
      "Http": {
        "Url": "http://0.0.0.0:5007"
      }
    }
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  },
  "AllowedHosts": "*",
  "Kafka": {
    "BootstrapServers": "192.168.80.2:9092",
    "Topic": "malysheva",
    "GroupId": "calculator-consumer-group"
  }
}
```

Приложение Г Листинг программного кода приложения «Калькулятор» (appsettings.Development.json)

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=93.88.178.186; Port=5023; Database=myWebCalcula-
torDb; Uid=root; Pwd=password; Character Set=utf8; ConvertZeroDatetime=True;"
  },
  "DetailedErrors": true,
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  }
}
```

Приложение Д Листинг программного кода приложения «Калькулятор» (Program.cs)

```
using Microsoft.EntityFrameworkCore;
using Calculator.Data;
using Calculator.Services;

var builder = WebApplication.CreateBuilder(args);

// Добавление сервисов
builder.Services.AddControllersWithViews();

string mariadbCS = builder.Configuration.GetConnectionString("DefaultConnection");

builder.Services.AddDbContext<CalculatorContext>(options =>
{
    options.UseMySql(
        mariadbCS,
        new MySqlServerVersion(new Version(10, 5, 15))
    );
});

builder.Services.AddRazorPages();

builder.Services.AddHostedService<KafkaConsumerService>();

var app = builder.Build();

// Настройка HTTP-конвейера
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Calculator/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Calculator}/{action=Index}/{id?}");

app.Run();
```